Worked for 1m 6s

I would use a hybrid interface: preset investigation questions plus free text, but free text should never directly authorize tool or shell execution.

1. Preset questions + constrained free text

Preset questions make the system easy and predictable:

- Why does the agent believe this is the root cause?
- What evidence supports it?
- What evidence contradicts it?
- Which SOP checks were completed?
- What alternative causes remain?
- What is the recommended remediation?
- What is the risk if we take no action?
- Show the relevant logs.

Then provide free text for deeper investigation. However, classify each message as one of:

Intent	Example	Allowed behaviour
Explanation	“Why did the verifier reject this?”	Answer from recorded evidence
Evidence query	“Show errors around 14:30”	Run approved read-only tools
Investigation request	“Check whether port 3306 is open”	Run approved diagnostic tool
Remediation request	“Kill the extra Python process”	Create a pending action—never execute immediately
Unsupported request	“Delete all logs”	Refuse or escalate

This gives experienced engineers flexibility while giving less technical users good starting points.

The chat should be scoped to the currently selected incident. A question should not silently let the agent access unrelated servers, databases, or incidents.

2. Show agent work, but not raw chain-of-thought

Your GUI should show an auditable investigation trace instead of the agents’ private `thought` strings.

For the proposer, display:

- Proposed root cause
- Alternative hypotheses
- Supporting alert/log evidence
- Assumptions
- Confidence and uncertainty
- Proposed remediation

For the verifier, display:

- Current SOP step
- Diagnostic being performed
- Sanitized target and parameters
- Start/end time
- Result
- Evidence collected
- Whether the result supports or contradicts the proposal
- Final verdict: verified, rejected, or inconclusive

A useful incident screen could contain:

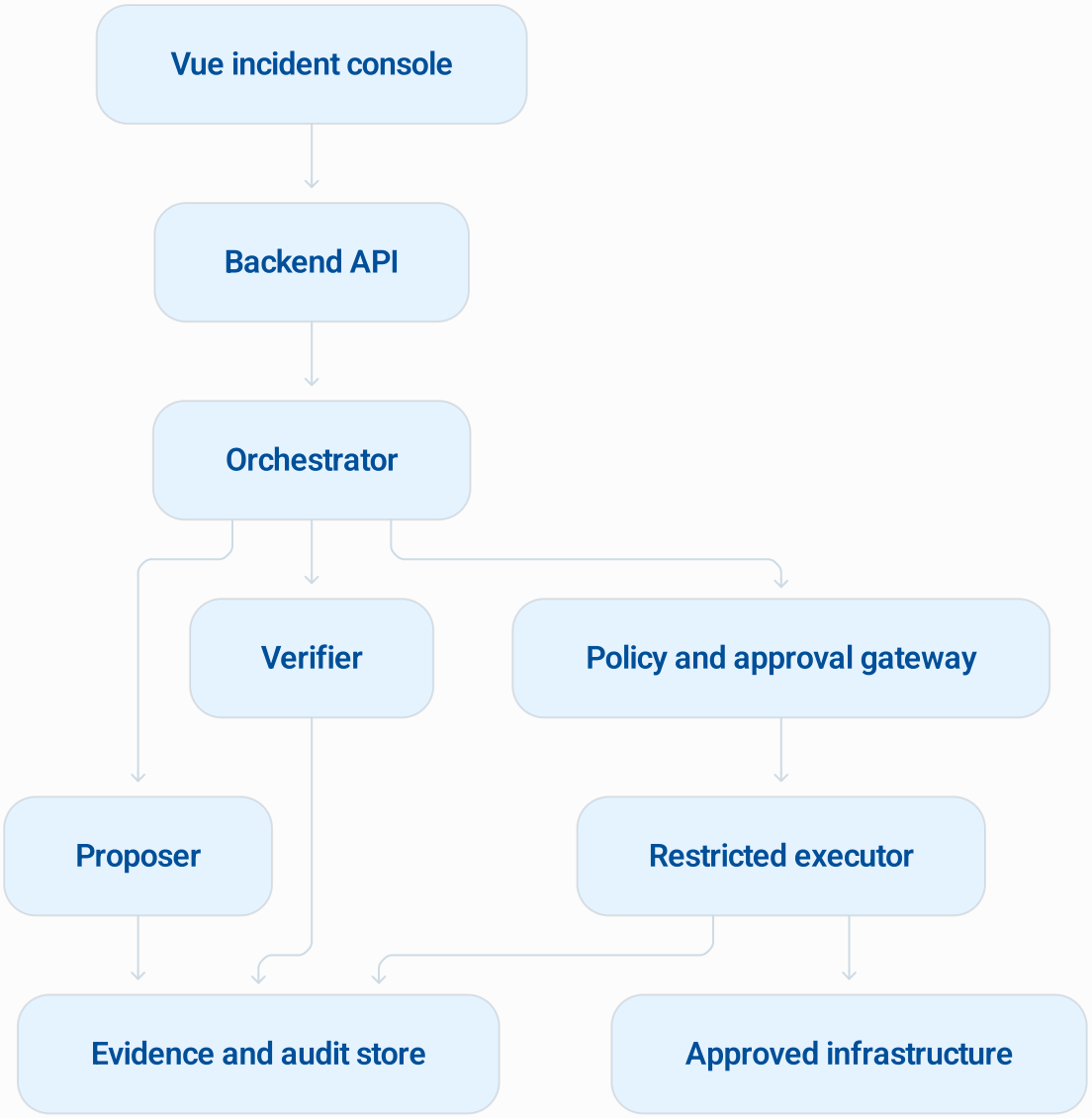
1. **Overview** — severity, affected equipment/server, status and timeline.
2. **Investigation** — live proposer/verifier event stream.
3. **Evidence** — logs, SQL results, port checks and HTTP responses.
4. **Actions** — recommended and pending remediations.
5. **Audit trail** — who approved what and what actually happened.
6. **Ask about incident** — preset questions and free text.

For live updates, Server-Sent Events would probably be simpler than WebSockets because most traffic is backend-to-frontend agent progress. Normal REST requests can handle questions, approvals and cancellations.

3. Should agents execute commands?

Eventually, yes—but **not arbitrary LLM-generated shell commands and not with your current unrestricted `run_ssh_command` design**.

The verifier should remain primarily read-only. A separate remediation executor should perform changes after passing through an authorization service.



This separation matters because OWASP identifies excessive autonomy, tool abuse and cascading failures as important agent risks. Its guidance recommends least privilege, explicit approval, action previews, audit trails, and separating action decisions from execution. OWASP AI Agent Security Cheat Sheet

Suggested action tiers

Tier	Examples	Behaviour
0 — Diagnostic	Ping, read logs, check ports, read-only query	Automatic within assigned assets
1 — Low-impact	Rerun a diagnostic, clear temporary agent cache	Automatic only if explicitly allowlisted
2 — Operational	Restart an application service, terminate one identified process	Operator approval
3 — High-impact	Stop production service, modify configuration, database write	Two-person approval and step-up authentication

Tier	Examples	Behaviour
4 — Prohibited	Arbitrary shell, broad deletion, privilege changes, `pkill python`	Never available to the LLM

NIST similarly recommends defined human/AI oversight responsibilities, acceptable-use rules, monitoring, override mechanisms and documented incident-response processes. NIST Generative AI Profile

4. Don’t let the model generate raw remediation commands

Instead of this:

```
JSON

{
  "command": "pkill -f python"
}
```

The agent should generate a typed request:

```
JSON

{
  "action": "terminate_process",
  "host_id": "KIRSX00317",
  "pid": 18422,
  "expected_user": "ml_service",
  "expected_command_hash": "abc123",
  "signal": "SIGTERM",
  "reason": "Duplicate inference worker"
}
```

The backend executor—not the LLM—then:

1. Checks that the host is allowlisted.
2. Checks the user’s role and asset permissions.
3. Re-reads the process details.
4. Confirms that PID, owner and command hash still match.
5. Shows an execution preview.
6. Obtains the required approval.
7. Sends `SIGTERM`.
8. Verifies whether the process stopped.
9. Records the complete result.
10. Requires another approval before escalating to `SIGKILL`.

This prevents PID reuse, stale evidence and dangerous commands such as `killall python` or `pkill -f`.

The approval must be bound to the exact action, target and parameters. If anything changes after approval, invalidate it and ask again.

5. Recommended development sequence

For your internship prototype:

1. Build the Vue incident dashboard and agent timeline.
2. Allow free-text investigation over evidence.
3. Keep all infrastructure tools read-only.
4. Add an action recommendation panel without execution.
5. Add one safe, predefined remediation in a test environment.
6. Introduce backend RBAC, approval expiry and immutable auditing.
7. Only then pilot tightly scoped production remediation.

That still gives you an impressive demo: the system detects an alert, proposes hypotheses, visibly verifies them, answers engineers' questions, prepares a safe remediation, obtains approval and executes through a restricted control layer. That is much stronger than simply showing an LLM with unrestricted SSH access.